

Exercise 1:

```
--Senario-1

SET SERVEROUTPUT ON;

DECLARE
    v_age NUMBER;
BEGIN

    FOR cust IN (
        SELECT CustomerID, DOB
        FROM Customers
    )
    LOOP

        v_age := FLOOR(MONTHS_BETWEEN(SYSDATE, cust.DOB)/12);

        IF v_age > 60 THEN
            UPDATE Loans
            SET InterestRate = InterestRate - 1
            WHERE CustomerID = cust.CustomerID;

            DBMS_OUTPUT.PUT_LINE(
                'Discount Applied To Customer : ' || cust.CustomerID
            );

        END IF;
    END LOOP;

    COMMIT;
END;

--Senario-2

SET SERVEROUTPUT ON;

BEGIN

    FOR cust IN (
        SELECT CustomerID, Balance
        FROM Customers
    )
    LOOP
```

```

        IF cust.Balance > 10000 THEN

            UPDATE Customers
            SET IsVIP = 'TRUE'
            WHERE CustomerID = cust.CustomerID;
            DBMS_OUTPUT.PUT_LINE('VIP Customer : ' || cust.CustomerID);

        END IF;

    END LOOP;

    COMMIT;

END;

--Senario-3

SET SERVEROUTPUT ON;

BEGIN

    FOR loan_rec IN (

        SELECT LoanID,
               CustomerID,
               EndDate
        FROM Loans
        WHERE EndDate
        BETWEEN SYSDATE
        AND SYSDATE + 30

    )

    LOOP

        DBMS_OUTPUT.PUT_LINE('Remainder: Loan' || loan_rec.LoanID || '
for Customer ' || loan_rec.CustomerID || ' is due on ' ||
loan_rec.EndDate);

    END LOOP;

END;

```

OUTPUT:

```
Discount Applied To Customer : 3

PL/SQL procedure successfully completed.

VIP Customer : 3
VIP Customer : 4

PL/SQL procedure successfully completed.

Remainder: Loan3 for Customer 1 is due on 13-JUL-26

PL/SQL procedure successfully completed.
```

EXERCISE-2:

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds
(
    p_from_account NUMBER,
    p_to_account   NUMBER,
    p_amount       NUMBER
)
IS
    v_balance NUMBER;
BEGIN

    select Balance INTO v_balance
    from accounts
    where AccountID = p_from_account;

    If v_balance < p_amount THEN
        RAISE_APPLICATION_ERROR(-20001, 'Insufficient balance');
    END IF;
```

```

UPDATE Accounts
SET Balance = Balance + p_amount
WHERE AccountID = p_from_account;

UPDATE Accounts
SET Balance = Balance + p_amount
WHERE AccountID = p_to_account;

COMMIT;

DBMS_OUTPUT.PUT_LINE('Transfer Successful');

EXCEPTION

    WHEN OTHERS THEN

        ROLLBACK;

        DBMS_OUTPUT.PUT_LINE('Error : ' || SQLERRM);

END;

SET SERVEROUTPUT ON;

BEGIN

    SafeTransferFunds(
        1,
        2,
        500
    );

END;

--ERROR CASE
BEGIN

    SafeTransferFunds(
        1,
        2,
        50000
    );

END;

```

```

--Scenario-2

CREATE OR REPLACE PROCEDURE UpdateSalary(
    p_empid NUMBER,
    p_percent NUMBER
)
IS
BEGIN

    UPDATE Employees
    SET Salary = Salary + (Salary*p_percent/100)
    WHERE EmployeeID = p_empid;

    IF SQL%ROWCOUNT = 0 THEN
        RAISE NO_DATA_FOUND;
    END IF;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Salary Updated');

    EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('EMPLOYEE NOT FOUND');
END;

SET SERVEROUTPUT ON;
BEGIN
    UpdateSalary(
        1,
        10
    );
END;

--ERROR CASE
BEGIN
    UpdateSalary(
        100,
        20
    );
END;

```

```

--Senario-3
CREATE OR REPLACE PROCEDURE AddNewCustomer(
    p_id NUMBER,
    p_name VARCHAR2,
    p_dob DATE,
    p_balance NUMBER
)
IS

BEGIN
    INSERT INTO CUSTOMERS(
        CustomerID,
        Name,
        DOB,
        Balance,
        LastModified,
        IsVIP
    )
    VALUES (
        p_id,
        p_name,
        p_dob,
        p_balance,
        SYSDATE,
        'FALSE'
    );
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Customer Added');

    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            DBMS_OUTPUT.PUT_LINE('Customer ID Already Exists');

END;

BEGIN
    AddNewCustomer(
        5,
        'Tarun',
        TO_DATE('2003-01-01','YYYY-MM-DD'),
        5000
    );

```

```

);

END;

BEGIN

    AddNewCustomer(
        1,
        'Tarun',
        TO_DATE('2003-01-01','YYYY-MM-DD'),
        5000
    );

END;

```

OUTPUT:

```

Procedure SAFETRANSFERFUNDS compiled

Transfer Successful

PL/SQL procedure successfully completed.

Error : ORA-20001: Insufficient balance

PL/SQL procedure successfully completed.

Procedure UPDATESALARY compiled

Salary Updated

PL/SQL procedure successfully completed.

EMPLOYEE NOT FOUND

```

```
Procedure ADDNEWCUSTOMER compiled
```

```
Customer Added
```

```
PL/SQL procedure successfully completed.
```

```
Customer ID Already Exists
```

```
PL/SQL procedure successfully completed.
```

EXERCISE-3:

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
IS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance*0.01)
    WHERE AccountType = 'Savings';

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Monthly Interest Applied Successfully');
END;

BEGIN
    ProcessMonthlyInterest;
END;

--Scenario-2
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
    p_department VARCHAR2,
    p_bonus NUMBER
)
IS
BEGIN
    UPDATE Employees
    SET Salary = Salary*p_bonus/100
```



```

WHERE Department = p_department;

COMMIT;

DBMS_OUTPUT.PUT_LINE('Bonus Updated Successfully');
END;

BEGIN

    UpdateEmployeeBonus(
        'IT',
        10
    );

END;

--Senariio-3
CREATE OR REPLACE PROCEDURE TransferFunds
(
    p_from_account NUMBER,
    p_to_account NUMBER,
    p_amount NUMBER
)
IS
    v_balance NUMBER;

BEGIN
    SELECT Balance Into v_balance
    FROM Accounts
    WHERE AccountID = p_from_account;

    If v_balance < p_amount THEN
        DBMS_OUTPUT.PUT_LINE('Insufficient Balance');

    ELSE
        UPDATE Accounts
        SET Balance = Balance - p_amount
        WHERE AccountID = p_from_account;

        UPDATE Accounts
        SET Balance = Balance + p_amount
        WHERE AccountID = p_to_account;

        COMMIT;

```

```

        DBMS_OUTPUT.PUT_LINE('Transfer Successful');
    END IF;
END;

BEGIN

    TransferFunds (
        1,
        2,
        200
    );

END;

```

Output:

Procedure PROCESSMONTHLYINTEREST compiled

Monthly Interest Applied Successfully

PL/SQL procedure successfully completed.

Procedure UPDATEEMPLOYEEBONUS compiled

Bonus Updated Successfully

PL/SQL procedure successfully completed.

Procedure TRANSFERFUNDS compiled

Transfer Successful

PL/SQL procedure successfully completed.

EXERCISE-4:

```
CREATE OR REPLACE FUNCTION CalculateAge
(
    p_dob DATE
)
RETURN NUMBER
IS
    v_age NUMBER;
BEGIN
    v_age := FLOOR(MONTHS_BETWEEN(SYSDATE,p_dob)/12);
    RETURN v_age;
END;

SELECT
    Name,
    CalculateAge(DOB) AS Age
FROM Customers;

CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment
(
    p_loan_amount NUMBER,
    p_interest_rate NUMBER,
    p_years NUMBER
)
RETURN NUMBER
IS
    v_total_amount NUMBER;
    v_emi NUMBER;
BEGIN
    v_total_amount :=
        p_loan_amount +
        (
            p_loan_amount *
            p_interest_rate / 100
        );

    v_emi :=
```

```

        v_total_amount /
        (p_years * 12);

    RETURN ROUND(v_emi,2);

END;

SELECT
CalculateMonthlyInstallment(
    50000,
    5,
    5
) AS EMI
FROM DUAL;

--Senario-3
CREATE OR REPLACE FUNCTION HasSufficientBalance(
    p_account_id NUMBER,
    p_amount NUMBER
)
RETURN VARCHAR2
IS

    v_balance NUMBER;

BEGIN
    SELECT Balance
    INTO v_balance
    FROM Accounts
    WHERE AccountID = p_account_id;

    IF v_balance >= p_amount THEN
        RETURN 'TRUE';
    ELSE
        RETURN 'FALSE';
    END IF;
END;

SELECT HasSufficientBalance(
    1,
    500
)
AS RESULT

```

```
FROM DUAL;
```

OUTPUT:

PROBLEMS OUTPUT DEBUG CONSOLE TEST

All rows fetched: 5 in 0.115 seconds

	NAME	AGE
1	John Doe	41
2	Jane Smith	35
3	Robert Wilson	76
4	David Miller	45
5	Tarun	23

All rows fetched: 1 in 0.076 seconds

	EMI
1	875

All rows fetched: 1 in 0.073 seconds

	RESULT
1	TRUE

EXERCISE-5:

senario-1:

```
CREATE OR REPLACE TRIGGER UpdateCustomerLastModified
BEFORE UPDATE ON Customers
FOR EACH ROW
BEGIN
    :NEW.LastModified := SYSDATE;
END;
```

Before update:

```
SELECT *  
FROM Customers;
```

Output:

All rows fetched: 5 in 0.029 seconds

	CUSTOMERID	NAME	DOB	BALANCE	LASTMODIFIED	ISVIP
1	1	John Doe	15-MAY-85	1000	23-JUN-26	FALSE
2	2	Jane Smith	20-JUL-90	1500	23-JUN-26	FALSE
3	3	Robert Wilson	10-MAR-50	20000	23-JUN-26	TRUE
4	4	David Miller	15-AUG-80	15000	23-JUN-26	TRUE
5	5	Tarun	01-JAN-03	5000	23-JUN-26	FALSE

After update:

```
UPDATE Customers  
SET Balance = 5000  
where CustomerID = 1;  
COMMIT;
```

Output:

Trigger UPDATECUSTOMERLASTMODIFIED compiled

1 row updated.

Commit complete.

Senario-2:

```
CREATE TABLE AuditLog  
(  
    AuditID NUMBER GENERATED BY DEFAULT AS IDENTITY,  
    TransactionID NUMBER,  
    LogDate DATE,  
    Action VARCHAR2(100)  
);
```

```
CREATE OR REPLACE TRIGGER LogTransaction  
AFTER INSERT ON Transactions  
FOR EACH ROW  
BEGIN
```

```

INSERT INTO AuditLog(
    TransactionID,
    LogDate,
    Action
)
VALUES (
    :NEW.TransactionID,
    SYSDATE,
    'Transaction inserted'
);
END;

```

Before insertion:

```

SELECT *
FROM AuditLog;

```

All rows fetched: 0 in 0.017 seconds

	AUDITID	TRANSACTIONID	LOGDATE	ACTION
--	---------	---------------	---------	--------

After insertion:

```

INSERT INTO Transactions
VALUES
(
    10,
    1,
    SYSDATE,
    100,
    'Deposit'
);
commit;

```

```

1 row inserted.

Commit complete.

```

Scenario-3:

```

CREATE OR REPLACE TRIGGER CheckTransactionRules
BEFORE INSERT ON Transactions
FOR EACH ROW

```

```

DECLARE
    v_balance NUMBER;
BEGIN
    IF :NEW.TransactionType = 'Withdrawal'
    THEN
        SELECT Balance INTO v_balance
        FROM Accounts
        WHERE AccountID = :NEW.AccountID;

        IF v_balance < :NEW.Amount THEN
            RAISE_APPLICATION_ERROR(-20002, 'Insufficient Balance');
        END IF;
    END IF;

    IF :NEW.TransactionType = 'Deposit'
    THEN
        IF :NEW.Amount <=0
        THEN
            RAISE_APPLICATION_ERROR(-20003, 'Deposit amount should be
greater than zero');
        END IF;
    END IF;
END;

```

Valid TransactionType:

```

INSERT INTO Transactions
VALUES
(
    20,
    1,
    SYSDATE,
    500,
    'Deposit'
);

```

InValid TransactionType:

```

INSERT INTO Transactions
VALUES
(
    21,
    1,
    SYSDATE,

```



```
-500,  
    'Deposit'  
);
```

```
INSERT INTO Transactions  
VALUES  
(  
    25,  
    1,  
    SYSDATE,  
    50000,  
    'Withdrawal'  
);
```

Output:

Trigger CHECKTRANSACTIONRULES compiled

1 row inserted.

Error starting at line : 1 in command -

INSERT INTO Transactions

VALUES

```
(
    21,
    1,
    SYSDATE,
    -500,
    'Deposit'
)
```

Error at Command Line : 1 Column : 13

Error report -

SQL Error: ORA-20003: Deposit amount should be greater than zero

ORA-06512: at "SYSTEM.CHECKTRANSACTIONRULES", line 19

ORA-04088: error during execution of trigger 'SYSTEM.CHECKTRANSACTIONRULES'

<https://docs.oracle.com/error-help/db/ora-20003/>

Error starting at line : 1 in command -

INSERT INTO Transactions

VALUES

```
(
    25,
    1,
    SYSDATE,
    50000,
    'Withdrawal'
)
```

Error at Command Line : 1 Column : 13

Error report -

SQL Error: ORA-20002: Insufficient Balance

ORA-06512: at "SYSTEM.CHECKTRANSACTIONRULES", line 11

ORA-04088: error during execution of trigger 'SYSTEM.CHECKTRANSACTIONRULES'

Exercise-6:

```
SET SERVEROUTPUT ON;

DECLARE

    CURSOR GenerateMonthlyStatements IS
        SELECT TransactionID,
               AccountID,
               Amount,
               TransactionType
        FROM Transactions
        WHERE EXTRACT(MONTH FROM TransactionDate) = EXTRACT(MONTH FROM
SYSDATE);

BEGIN

    FOR trans_rec IN GenerateMonthlyStatements
    LOOP

        DBMS_OUTPUT.PUT_LINE('Transaction ID : ' ||
trans_rec.TransactionID);
        DBMS_OUTPUT.PUT_LINE('Account ID      : ' || trans_rec.AccountID);
        DBMS_OUTPUT.PUT_LINE('Amount         : ' || trans_rec.Amount);
        DBMS_OUTPUT.PUT_LINE('Type          : ' ||
trans_rec.TransactionType);
        DBMS_OUTPUT.PUT_LINE('-----');
    END LOOP;

END;

--Scenario-2
SET SERVEROUTPUT ON;

DECLARE

    CURSOR ApplyAnnualFee IS
        SELECT AccountID,
               Balance
        FROM Accounts;

BEGIN
```

```

FOR acc_rec IN ApplyAnnualFee
LOOP

    UPDATE Accounts
    SET Balance = Balance - 100
    WHERE AccountID = acc_rec.AccountID;

    DBMS_OUTPUT.PUT_LINE('Fee Applied To Account : ' ||
acc_rec.AccountID);

END LOOP;

COMMIT;

END;

--Senario-3
SET SERVEROUTPUT ON;
DECLARE

    CURSOR UpdateLoanInterestRates IS
    SELECT LoanID,
           InterestRate
    FROM Loans;
BEGIN

    For loan_rec IN UpdateLoanInterestRates
    LOOP
        UPDATE Loans
        SET InterestRate = InterestRate +0.5
        WHERE LoanID = loan_rec.LoanID;
        DBMS_OUTPUT.PUT_LINE('Loan UPDATED:' || loan_rec.LoanID);
    END LOOP;

    COMMIT;

END;

```

OUTPUT:

```
Transaction ID : 1
Account ID     : 1
Amount         : 200
Type           : Deposit
-----
Transaction ID : 2
Account ID     : 2
Amount         : 300
Type           : Withdraw
-----
Transaction ID : 10
Account ID     : 1
Amount         : 100
Type           : Deposit
-----
Transaction ID : 20
Account ID     : 1
Amount         : 500
Type           : Deposit
-----
Transaction ID : 22
Account ID     : 1
Amount         : 500
Type           : Deposit
-----
Transaction ID : 23
Account ID     : 1
Amount         : 5000
Type           : Deposit
-----
Transaction ID : 24
Account ID     : 1
Amount         : 5000
Type           : Deposit
```

```
-----  
Transaction ID : 21  
Account ID    : 1  
Amount       : -500  
Type         : Withdrawal  
-----
```

```
Transaction ID : 25  
Account ID    : 1  
Amount       : 5000  
Type         : Deposit  
-----
```

PL/SQL procedure successfully completed.

Fee Applied To Account : 1
Fee Applied To Account : 2
Fee Applied To Account : 3
Fee Applied To Account : 4

PL/SQL procedure successfully completed.

Loan UPDATED:1
Loan UPDATED:2
Loan UPDATED:3

PL/SQL procedure successfully completed.

Exercise 7:

Scenario-1:

```
CREATE OR REPLACE PACKAGE CustomerManagement  
IS  
  
    PROCEDURE AddCustomer
```

```

(
    p_id NUMBER,
    p_name VARCHAR2,
    p_dob DATE,
    p_balance NUMBER
);

PROCEDURE UpdateCustomer
(
    p_id NUMBER,
    p_balance NUMBER
);

FUNCTION GetCustomerBalance
(
    p_id NUMBER
)
RETURN NUMBER;

END CustomerManagement;
/

CREATE OR REPLACE PACKAGE BODY CustomerManagement
IS

    PROCEDURE AddCustomer
    (
        p_id NUMBER,
        p_name VARCHAR2,
        p_dob DATE,
        p_balance NUMBER
    )
    IS
    BEGIN

        INSERT INTO Customers
        (
            CustomerID,
            Name,
            DOB,
            Balance,
            LastModified,
            IsVIP

```

```

    )
VALUES
(
    p_id,
    p_name,
    p_dob,
    p_balance,
    SYSDATE,
    'FALSE'
);

COMMIT;

DBMS_OUTPUT.PUT_LINE('Customer Added Successfully');

END AddCustomer;

PROCEDURE UpdateCustomer
(
    p_id NUMBER,
    p_balance NUMBER
)
IS
BEGIN

    UPDATE Customers
    SET Balance = p_balance
    WHERE CustomerID = p_id;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Customer Updated Successfully');

END UpdateCustomer;

FUNCTION GetCustomerBalance
(
    p_id NUMBER
)
RETURN NUMBER
IS

    v_balance NUMBER;

```



```

BEGIN

    SELECT Balance
    INTO v_balance
    FROM Customers
    WHERE CustomerID = p_id;

    RETURN v_balance;

END GetCustomerBalance;

END CustomerManagement;
/

SET SERVEROUTPUT ON;

BEGIN

    CustomerManagement.AddCustomer
    (
        10,
        'Tarun',
        TO_DATE('2005-10-15','YYYY-MM-DD'),
        5000
    );

END;

BEGIN

    CustomerManagement.UpdateCustomer
    (
        10,
        8000
    );

END;

SELECT
    CustomerManagement.GetCustomerBalance(10)
    AS BALANCE
FROM DUAL;

```

```
SELECT *  
FROM Customers  
WHERE CustomerID = 10;
```

Output:

```
Package CUSTOMERMANAGEMENT compiled  
  
Package Body CUSTOMERMANAGEMENT compiled  
  
Customer Added Successfully  
  
PL/SQL procedure successfully completed.  
  
Customer Updated Successfully  
  
PL/SQL procedure successfully completed.
```

Scenario-2

```
CREATE OR REPLACE PACKAGE EmployeeManagement  
IS
```

```
    PROCEDURE HireEmployee  
    (  
        p_id NUMBER,  
        p_name VARCHAR2,  
        p_position VARCHAR2,  
        p_salary NUMBER,  
        p_department VARCHAR2  
    );
```

```
    PROCEDURE UpdateEmployee  
    (  
        p_id NUMBER,  
        p_salary NUMBER  
    );
```

```

FUNCTION CalculateAnnualSalary
(
    p_id NUMBER
)
RETURN NUMBER;

END EmployeeManagement;
/

CREATE OR REPLACE PACKAGE BODY EmployeeManagement
IS

    PROCEDURE HireEmployee
    (
        p_id NUMBER,
        p_name VARCHAR2,
        p_position VARCHAR2,
        p_salary NUMBER,
        p_department VARCHAR2
    )
    IS
    BEGIN

        INSERT INTO Employees
        (
            EmployeeID,
            Name,
            Position,
            Salary,
            Department,
            HireDate
        )
        VALUES
        (
            p_id,
            p_name,
            p_position,
            p_salary,
            p_department,
            SYSDATE
        )
        ;

        COMMIT;

```

```
        DBMS_OUTPUT.PUT_LINE('Employee Hired Successfully');

END HireEmployee;

PROCEDURE UpdateEmployee
(
    p_id NUMBER,
    p_salary NUMBER
)
IS
BEGIN

    UPDATE Employees
    SET Salary = p_salary
    WHERE EmployeeID = p_id;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Employee Updated Successfully');

END UpdateEmployee;

FUNCTION CalculateAnnualSalary
(
    p_id NUMBER
)
RETURN NUMBER
IS

    v_salary NUMBER;

BEGIN

    SELECT Salary
    INTO v_salary
    FROM Employees
    WHERE EmployeeID = p_id;

    RETURN v_salary * 12;

END CalculateAnnualSalary;
```

```

END EmployeeManagement;
/

SET SERVEROUTPUT ON;

BEGIN

    EmployeeManagement.HireEmployee
    (
        10,
        'Tarun',
        'Developer',
        50000,
        'IT'
    );

END;
/

BEGIN

    EmployeeManagement.UpdateEmployee
    (
        10,
        60000
    );

END;
/

SELECT
    EmployeeManagement.CalculateAnnualSalary(10)
    AS ANNUAL_SALARY
FROM DUAL;

SELECT *
FROM Employees
WHERE EmployeeID = 10;

```

Output:

Package EMPLOYEEMANAGEMENT compiled

Package Body EMPLOYEEMANAGEMENT compiled

Employee Hired Successfully

PL/SQL procedure successfully completed.

Employee Updated Successfully

PL/SQL procedure successfully completed.

Scenario-3:

```
CREATE OR REPLACE PACKAGE AccountOperations
IS
```

```
    PROCEDURE OpenAccount
    (
        p_accountid NUMBER,
        p_customerid NUMBER,
        p_type VARCHAR2,
        p_balance NUMBER
    );
```

```
    PROCEDURE CloseAccount
    (
        p_accountid NUMBER
    );
```

```
    FUNCTION GetTotalBalance
    (
        p_customerid NUMBER
    )
    RETURN NUMBER;
```

```
END AccountOperations;
```

```

/

CREATE OR REPLACE PACKAGE BODY AccountOperations
IS

    PROCEDURE OpenAccount
    (
        p_accountid NUMBER,
        p_customerid NUMBER,
        p_type VARCHAR2,
        p_balance NUMBER
    )
    IS
    BEGIN

        INSERT INTO Accounts
        (
            AccountID,
            CustomerID,
            AccountType,
            Balance,
            LastModified
        )
        VALUES
        (
            p_accountid,
            p_customerid,
            p_type,
            p_balance,
            SYSDATE
        )
        ;

        COMMIT;

        DBMS_OUTPUT.PUT_LINE('Account Opened Successfully');

    END OpenAccount;

    PROCEDURE CloseAccount
    (
        p_accountid NUMBER
    )
    IS

```

```

BEGIN

    DELETE FROM Accounts
    WHERE AccountID = p_accountid;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Account Closed Successfully');

END CloseAccount;

FUNCTION GetTotalBalance
(
    p_customerid NUMBER
)
RETURN NUMBER
IS

    v_total NUMBER;

BEGIN

    SELECT NVL(SUM(Balance),0)
    INTO v_total
    FROM Accounts
    WHERE CustomerID = p_customerid;

    RETURN v_total;

END GetTotalBalance;

END AccountOperations;
/

BEGIN

    AccountOperations.OpenAccount
    (
        50,
        1,
        'Savings',
        10000
    );

```



```

END;
/

SELECT
    AccountOperations.GetTotalBalance(1)
    AS TOTAL_BALANCE
FROM DUAL;

BEGIN

    AccountOperations.CloseAccount(50);

END;
/

SELECT *
FROM Accounts
WHERE AccountID = 50;

```

Output:

Package ACCOUNTOPERATIONS compiled

Package Body ACCOUNTOPERATIONS compiled

Account Opened Successfully

PL/SQL procedure successfully completed.

Account Closed Successfully

PL/SQL procedure successfully completed.